

计算机系统导论实验报告

**实验名称: 程序链接 - 重定位
(IA-32)**

班级: 计算 2403

姓名: 钟智勇

日期: 2026 年 1 月 7 日

目录

1 实验报告四：程序链接 - 重定位 (IA-32)	3
1.0.1 1. 实验任务与目标	3
1.0.2 2. 完成该实验需要掌握的知识	3
1.0.3 3. 实验过程记录	3
1.0.3.1 3.1 识别“被抹除”的重定位记录	3
1.0.3.2 3.2 建立符号映射表	3
1.0.3.3 3.3 构造二进制补丁	3
1.0.3.4 3.4 十六进制修复	4
1.0.3.5 3.5 链接与验证	4
1.0.4 4. 遇到的问题与解决过程	4
1.0.5 5. 心得体会	4

1 实验报告四：程序链接 - 重定位 (IA-32)

1.0.1 1. 实验任务与目标

本实验的任务是修复一个“受损”的可重定位目标文件 `phase1.o`。该文件中的重定位节 (`.rel.text` 或 `.rel.data`) 部分记录被清零，导致无法与 `main.o` 正确链接，或者链接后运行结果错误。

-实验目标：

- 理解 ELF 格式中 重定位记录 (Elf32_Rel) 的二进制构成。
- 掌握 绝对寻址 (R_386_32) 与 相对寻址 (R_386_PC32) 的区别与应用场景。
- 通过反汇编分析代码逻辑，人工推断缺失的符号引用，并使用十六进制编辑器修复 `.o` 文件，最终使程序运行并正确输出目标字符串 `xWMJ;@ek>`。

1.0.2 2. 完成该实验需要掌握的知识

- 重定位记录结构：每个记录占 8 字节。前 4 字节是 `offset` (需修改位置的偏移)，后 4 字节包含 `symbol` (符号表索引，占 24 位) 和 `type` (重定位类型，占 8 位)。
- 重定位类型：
 - R_386_32：绝对地址重定位，多用于访问全局变量或跳转表地址。
 - R_386_PC32：相对地址重定位，计算公式通常为 `offset + PC`。常用于 `call` 指令。
- 符号表分析：利用 `readelf -s` 查看符号的索引 (Index)，这是构造重定位二进制记录的关键。
- 反汇编识别：识别 `e8 fc ff ff ff` 等占位指令。`e8` 是 `call` 的机器码，后跟的 `fc ff ff ff` (即 -4) 是典型的待重定位偏移占位符。

1.0.3 3. 实验过程记录

1.0.3.1 3.1 识别“被抹除”的重定位记录

首先通过 `readelf -r phase1.o` 查看重定位表。我发现原本应该存在的若干条记录现在显示为 `Offset: 0, Info: 0...` 或者直接消失。接着，我打开 `objdump -d phase1.o` 生成的反汇编文件 `phase1.s`，寻找那些明显需要地址但现在却是零或占位符的地方：

- `generate_code` 函数：`ff: a3 00 00 00 00` (存储到 `CODE`)
- `transform_code` 函数：`6a: 8b 04 85 00 00 00 00` (访问 `CODE_TRAN_ARRAY`)
- 函数调用处：多处 `call` 指令后的偏移量均为 `fc ff ff ff`。

1.0.3.2 3.2 建立符号映射表

为了修复这些记录，我使用 `readelf -s phase1.o` 提取了符号表，并记下了关键符号的索引值 (示例，实际以实验环境为准)：

- `CODE`: 索引 5
- `DICT`: 索引 8
- `transform_code`: 索引 12
- `puts`: 索引 15 (UND, 定义在外部库)

1.0.3.3 3.3 构造二进制补丁

以修复 `generate_code` 中对 `transform_code` 的调用为例：

1. 确定偏移 (Offset)：反汇编显示 `call` 指令在 `0x116`，操作数从 `0x117` 开始。

2. 确定类型 (Type): 函数调用使用 R_386_PC32 (类型值通常为 2)。
3. 确定符号 (Symbol): 查表知 transform_code 索引为 12 (十六进制 0x0c)。
4. 组合 Info 字段: 。
5. 最终 8 字节序列: 17 01 00 00 02 0c 00 00 (小端序)。

1.0.3.4 3.4 十六进制修复

使用 hexedit phase1.o 定位到 .rel.text 节的起始位置。通过 readelf -s 找到该节的文件偏移 (例如 0x4d0)。我依次将推导出的 8 字节记录覆盖那些全零的空位。修复点主要包括:

- 全局变量 CODE、DICT、BUFFER 的绝对地址访问。
- strlen、puts 以及模块内函数的相对跳转调用。

1.0.3.5 3.5 链接与验证

修复完成后, 执行链接: gcc -no-pie -o linkbomb main.o phase1.o 如果重定位修复错误, 链接器会报 relocation truncated to fit 或 undefined reference。经过调整补丁中的符号索引, 最终链接成功并输出 xWMJ;@ek>。

1.0.4 4. 遇到的问题与解决过程

- 问题: PC 相对寻址的计算错误。起初我不理解为什么 call 后的占位符是 fc ff ff ff (-4)。
 - 解决: 查阅文档得知, 链接器计算时会加上当前位置的偏移。由于指令 call 自身占 5 字节, 操作数占 4 字节, 计算相对地址时需要补回这个 4 字节的偏移量。
 - 问题: 符号表索引找错。因为 readelf -s 输出包含局部符号和节符号 (SECTION), 最开始我数错了 Index 导致重定位到了错误的函数上。
 - 解决: 后来我发现 readelf -r 的输出格式可以直接显示符号名称 (如果记录没被完全抹除的话)。对于完全抹除的, 我严格对比 readelf -s 第一列的数字, 确保二进制 Info 字段的高 24 位准确无误。
-

1.0.5 5. 心得体会

- 链接器的“缝合”工作: 实验四让我真正看清了链接器是如何把一个个孤立的 .o 文件“缝合”成一个有机整体的。重定位表就是链接器的“缝合指南”。
- 机器码的严谨性: 在十六进制编辑器里修改一个字节, 就可能让程序从执行逻辑跳转到内存非法访问。这种“颗粒度”极细的操作让我对计算机系统的底层运行逻辑产生了敬畏。
- 静态分析的闭环: 从 C 代码框架出发, 到汇编指令寻找漏洞, 再到二进制重定位表进行修复。这个过程完成了一个从高级语言到底层实现的认知闭环, 极大地锻炼了我的逆向工程思维。